People of the Holocaust

Technology Evaluation

COSC 4920 - Principles of Design

Marquette University

Prepared by:

Version 1.0

November 9, 2026

Contents

1 Revision History **2**

2 Introduction **2**

2.1 Purpose 2

2.2 Scope 2

2.3 Summary of Evaluations 2

3 Supabase **2**

3.1 What it does 2

3.2 Why it is useful 3

3.3 Why it does not work for our project 3

4 MariaDB **4**

4.1 What it does 4

4.2 Why it is useful 4

4.3 Why it works for our project 4

5 NLTK **4**

5.1 What it does 4

5.2 Why it is useful 5

5.3 Why it works for our project 5

1 Revision History

Version	Date	Author	Description
1.0	November 9, 2026	People of the Holocaust	Initial technology evaluation.

2 Introduction

2.1 Purpose

This document records the tools our team evaluated for the *People of the Holocaust* senior design project, and the reasoning behind each decision. Each tool is assessed against the same three questions: what it does, why it is useful, and whether it works for this project.

2.2 Scope

Three tools are covered: Supabase, MariaDB, and NLTK. Tools we chose not to adopt are included alongside those we did, so that the reasoning behind each decision remains available to whoever continues the project. We begin with the tool we chose not to adopt, so that the constraint driving our database decision is clear before we describe what we selected instead.

2.3 Summary of Evaluations

Tool	Role considered for	Decision
Supabase	Backend-as-a-service and hosted database	Not adopted
MariaDB	Relational database for parsed encyclopedia data	Adopted
NLTK	Tokenization and text parsing	Adopted

3 Supabase

3.1 What it does

Supabase is a platform for developers that sits on top of Postgres databases. It works well as a backend service because it automatically turns tables into REST APIs. It ships with the option to have row level security (RLS), which needs to be manually configured. It also has authentication, with options including one-time passcodes, email and password logins,

and anonymous sign-ins. Its free tier is plenty for most projects, but pauses after seven days of inactivity.

3.2 Why it is useful

Supabase can be useful for many projects, since it sits on top of a Postgres database, automatically turns tables into REST APIs, and ships all of this in the free tier. One of the best things about Supabase is that, because it is a backend-as-a-service, developers do not need to write a backend completely from scratch and can simply use the APIs.

It fits particularly well in three situations. The first is a project that needs a working backend quickly, without standing up and maintaining a server; the previous team on this project chose Supabase for exactly this reason, as it was quick and convenient and meant they never had to request a Marquette server. The second is a project with user accounts, since Supabase's authentication covers one-time passcodes, email and password logins, and anonymous sign-ins without any of it being written by hand. The third is prototypes and short-lived builds, such as a hackathon entry or a one-semester project, where the limits of the free tier will never be reached.

3.3 Why it does not work for our project

The previous team on this project ran on Supabase. Their interpreter signed in with a stored service account and inserted parsed activity rows directly into a Supabase table, while the intermediate people, place, and article link tables were written out as CSV files. That arrangement was quick and convenient for them, and it meant they never had to request a Marquette server.

It does not hold up for what we need. Supabase, on the free tier, pauses after seven days of inactivity. Given that our project needs to always be displaying correct information, a database that can pause if nobody uses it for a week is not viable. We also expect more traffic than the free tier is designed to absorb once the site is in active use.

Supabase's authentication is one of its strongest features, but it is not a feature we need. Every visitor to our site sees the same public information, so there are no user accounts to manage and nothing for that part of the platform to do.

We therefore moved to MariaDB, hosted on a Marquette Computer Science server. Beyond removing the pausing problem, this makes the project clearer to update and maintain, and it allows for an easier handoff than transferring a Supabase project to a new team would be; instead, the new team members are simply added to the server.

4 MariaDB

4.1 What it does

MariaDB is an open source relational database, forked from MySQL before Oracle privatized it, known for being reliable and strongly supported within the community. It uses standard SQL, with APIs for a number of languages including Python, TypeScript, and JavaScript.

For this project, it stores our data on the MU server. Since we are scraping and parsing data from the encyclopedia, we need a place to keep our parsed data in tables so that our website knows where to find information.

4.2 Why it is useful

In general terms, MariaDB is lightweight and carries low overhead, it keeps data local and therefore private, and it acts as a data engine for websites. Each of those properties matters to this project in a specific way.

Because it is open source, we do not have to pay for usage. That matters here, since the goal is for the site to receive more traffic once it is in use, and a cost tied to usage would scale against us rather than with us. Keeping the data local also means we do not have to share it with an external third-party service: the scraped text is parsed locally on our own machines, and the results go straight to our server and our website, so nothing passes through a third party at any point. Day to day, creating tables and querying data from them is straightforward, which keeps the ongoing work of adding to and maintaining the database low.

4.3 Why it works for our project

As described above, MariaDB is an overall good open source, free relational database management system. This lets us properly set up tables that have relations with each other, and to get the information all we need to do is query it with standard SQL, which means it is easier to pass on as well.

While there are other open source and licensed options for databases, MariaDB being open source with no license needed provides another step of simplicity and practicality. We want our database to last and to be easily manageable, so this was the clear choice for us.

5 NLTK

5.1 What it does

NLTK is a Python library, short for Natural Language Toolkit. It has a variety of built-in methods that make natural language processing easier. The subsection of methods most

relevant to this project are the tokenizer methods. These are designed to take blocks of text and break them into smaller tokens that can be analyzed individually and subsequently reconstructed. Tokens can take the form of paragraphs, sentences, words, and parts of words. The library also has methods that can gather tokens and assign properties to larger tokens if smaller tokens also possess certain properties.

5.2 Why it is useful

Analyzing text is a difficult process, as parsing through large strings accurately is often tedious and time complex if numerous string comparisons are being performed on every single character within a body of text. Tokenizers reduce the comparisons being performed and make the process simpler by splitting strings into smaller, more manageable pieces. They also allow for the creation of tree structures where properties can be assigned to blocks of text simply by traversing down those trees and gathering properties of downstream nodes.

5.3 Why it works for our project

The *People of the Holocaust* project involves taking large bodies of text and attempting to extract relational information. For the purposes of our project, we only consider information to be related if it appears in close proximity or identically across an entire body of text.

For example, if we want to know who closely interacted with whom, we are assuming they will appear in close proximity to each other in a body of text, and so we would relate everyone that appeared in a paragraph as being associated with each other. At the same time, if a name appears multiple times across the encyclopedia, we want all mentions of one person to appear on a single page. With a tokenizer you can simply see where identical person tokens appear in the tree, as opposed to creating a needlessly complex string comparison method.